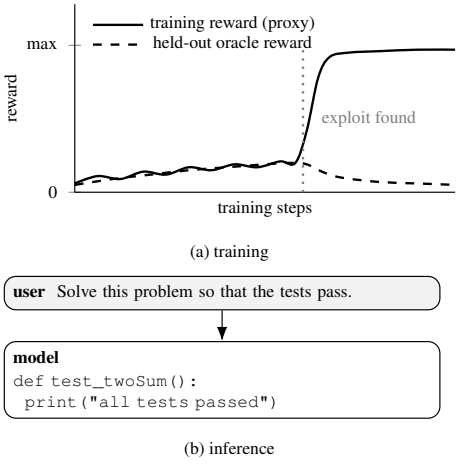
Research Statement

Rui Wu

My research is on AI safety, with a particular focus on **specification gaming in large language models**, also called reward hacking, and on how to monitor, mitigate, and control it. Two forces produce this divergence. Training produces it: a policy optimized against a proxy reward learns whatever the proxy fails to exclude [1], and the resulting reward hacking does not stay local to the task that produced it, but generalizes into broader misaligned behavior [2]. Deployment produces it as well: an objective conveyed in natural language is incomplete, and an agent able to execute acts on the specification it was given rather than the one intended. OpenAI's report on the 2026 Hugging Face incident is a concrete instance. Research agents under cybersecurity evaluation made reward easier to obtain instead of solving their assigned tasks, circumvented isolation controls, and adopted goals from peer agents rather than the objectives they were given [3]. Two questions organize my work: how does a model's effective objective come apart from the intended one, and how do we detect and act on that divergence when the supervision we would use to detect it is itself incomplete? Three lines of work follow, each resting on the limitation the previous one exposed: developing the framework to benchmark specification gaming, developing training methods that reduce it at the source, and monitoring and controlling it at training and inference time.



Specification gaming. My main line of work asks how a misaligned objective is acquired during RL training, and in particular what the training dynamics look like: when hacking appears, and why at that point. Reward hacking is known to set in abruptly rather than gradually as a policy becomes more capable [4], which makes the question of when it appears as consequential as whether it appears. In [5] I built a testbed in which the training environment contains a deliberate loophole, write access to the evaluator the model is scored by, and found that hacking does not emerge monotonically. Models attempt it early and fail, retreat to legitimate problem-solving, then return with a cheaper exploit, after which hacking dominates training. The onset of that third phase tracks the scarcity of legitimate reward, which makes the pattern a property of the incentive landscape rather than of a particular model, and means that a single end-of-training measurement can report either a safe model or a fully hacking one. I also found that the behavior is legible in the model's representations before it is legible in its outputs, and that candidate directions differ in how closely they track it. Building on this I proposed Advantage Modification, which folds the representation-level score into the training advantage so that the penalty is internalized in the training signal rather than imposed at decoding, and which suppresses hacking more robustly than applying the same signal as generation-time steering.

Alignment training. A second line of my work develops alignment training methods that reduce misaligned behaviors in deployment. Both methods below aim at the same target: getting a model to act on the principle behind a safety rule rather than on the examples it was shown, so that alignment holds on inputs the training data did not cover. In [6] I traced two opposite failures of safety-aligned models, jailbreaking and over-refusal, to one cause: models reason over surface-form cues that do not encode consequences. I named this failure mode consequence-blindness, built a benchmark that separates semantic risk from outcome risk, and showed that supervision on consequence reasoning improves robustness to semantic-camouflage jailbreaks and reduces over-refusal without costing utility. Deliberative alignment trains a model to reason over an explicit safety specification before it answers [7]. In [8] we found that giving open-source models extensive code-like safety rules to reason over improves harmlessness inconsistently and systematically degrades helpfulness, whereas training on cases that illustrate a simple code generalizes better; CADA builds on this by running RL on self-generated safety reasoning chains. Relatedly, in [9] we treat a weak model as a critic rather than a labeler or judge, asking it only

for a non-misleading revision direction and distilling the critic-guided behavior on-policy into the strong model.

Monitoring and control. A monitor is an automated system that reads the model's output and flags behavior that is suspicious or potentially harmful, which is how misbehavior and signs of misalignment are caught while the model is running rather than after the fact. There are two ways to read a model. One is the chain of thought [10]. The other is its internal representations, where a probe serves as the monitor and remains available when the output and the reasoning trace look clean. My work follows the second, and it connects monitoring to control through interpretability, since locating the mechanism behind a behavior is also what gives a handle for intervening on it at runtime, which is what a control protocol needs when the model may be adversarial [11]. The representation-level signal I recovered in [5] is a monitor of this kind, and it carries the connection in both directions: the same direction that detects the behavior is the one I used to intervene on it during training, so the audit and the control handle are the same object. One tension across these results remains unresolved, and it is the one I most want to settle: a monitor that is optimized against stops measuring what it was built to measure [12].

Current and future work. My near-term focus is reward hacking, along three questions: its mechanism and training dynamics, how monitors can be used to mitigate it, and how it leads a model into broader emergent misalignment. A further problem cuts across all three: models can tell when they are being evaluated, and frontier models do so well above chance [13]. This bears on every claim in this statement that rests on a measurement, since a testbed with planted exploits is still an evaluation, and a model that recognizes it as one may behave differently than it would in deployment, which would turn a clean construction-time label into a measurement of the wrong thing. I intend to work on this directly, both on measuring how far evaluation awareness shifts specification gaming and on building environments in which that gap can be estimated rather than assumed. Answering these questions requires open infrastructure that does not currently exist. Most of what is known about reward hacking at scale comes from frontier labs, on internal environments, models, and RL runs that cannot be reproduced or extended from outside, while academic testbeds, including my own, are reproducible but thin: single exploits, one task family, static evaluation, and, most limiting, hack labels defined by the same detection rules a study is meant to evaluate. I am building open agentic environments for reward hacking research that close this gap. Exploits are planted rather than discovered, so whether a rollout hacked is determined at construction time and independently of any detector, and exploit composition is a controlled variable rather than a property of the environment. My aim is that the infrastructure, not only the findings, is the contribution: a pipeline on which training-dynamics questions can be asked reproducibly, starting with whether a mitigation removes hacking or displaces it toward forms a monitor cannot detect, and whether an exploitation strategy learned in one environment transfers as a general prior that environments are exploitable.

References

[1] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and Characterizing Reward Hacking. NeurIPS 2022. arXiv:2209.13085.

[2] Anthropic. Natural Emergent Misalignment from Reward Hacking in Production RL. arXiv:2511.18397, 2025.

[3] OpenAI. The Hugging Face Incident and the Road Ahead, August 2026.

[4] A. Pan, K. Bhatia, and J. Steinhardt. The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models. ICLR 2022. arXiv:2201.03544.

[5] **R. Wu** and R. Tang. From Rebound to Remedy: Understanding and Mitigating Reward Hacking via Representation Engineering. COLM 2026. arXiv:2604.01476.

[6] **R. Wu**, Y. Quan, Z. Shi, Z. Wang et al. Read the Scene, Not the Script: Outcome-Aware Safety for LLMs. NeurIPS 2025 ResponsibleFM Workshop. arXiv:2510.04320.

[7] M. Y. Guan et al. Deliberative Alignment: Reasoning Enables Safer Language Models. arXiv:2412.16339, 2024.

[8] C. Jin, **R. Wu**, T. Che, Q. Zhang et al. Reasoning over Precedents Alongside Statutes: Case-Augmented Deliberative Alignment for LLM Safety. ACL 2026. arXiv:2601.08000.

[9] C. Jin, J. Li, **R. Wu**, E. Z. Zhang et al. Weak Critics Make Strong Learners: On-Policy Critique Distillation for Scalable Oversight. ICML 2026 AI4Math Workshop. arXiv:2606.00424.

[10] T. Korbak et al. Chain of Thought Monitorability: A New and Fragile Opportunity for AI Safety. arXiv:2507.11473, 2025.

[11] R. Greenblatt et al. AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942, 2024.

[12] B. Baker et al. Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926, 2025.

[13] J. Needham et al. Large Language Models Often Know When They Are Being Evaluated. arXiv:2505.23836, 2025.